UNIT IV: Inheritance & Pointers

Answer Key: Debugging and Pseudo-Code Formulation

Name: Kavitha B **Registration No:** 192565041

How to read this document. Section A answers the five debugging problems in the format required by the question bank: line numbers and defect category, the theoretical explanation, corrected compilable code, and the dry run with console output. Section B gives the five pseudo-code designs with dry-run trace tables.

Verification. Every corrected C++ program in Section A was compiled with g++ (-std=c++17 -Wall -Wextra) and run; the console outputs shown are the real outputs. The compiler messages quoted for the buggy code are the real g++ messages.

Line numbers. Lines are quoted as labelled by the // Line N comments in the question paper. g++ counts physical lines of the pasted code and can differ by a few lines; where it matters, the g++ line is given in brackets.

Small additions in corrected code. In a few programs I added print statements (for example in the constructors) or a virtual destructor so that the constructor order and polymorphic behaviour can be seen in the output. These additions are pointed out where they occur.

Section A: Debugging Questions

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Defect classification

Line	Defect	Category
33 (g++ line 35)	ta.idNumber is ambiguous	Compilation error: request for member 'idNumber' is ambiguous
12 and 18	Student and Employee inherit Person non-virtually	Root cause: multipath inheritance creates duplicate Person subobjects (Diamond Problem)

Task 1: The exact compilation error

g++ reports: error: request for member 'idNumber' is ambiguous (followed by notes that idNumber is found in both Person via Employee and Person via Student).

Task 2: Memory layout and why there are two idNumber fields

With ordinary (non-virtual) inheritance every base class is stored as a complete subobject inside the derived object. TeachingAssistant has two direct bases, Student and Employee, and each of them contains its own Person subobject. So the object holds two complete Persons:

Part of the TeachingAssistant object	Contents
Student subobject, containing a Person subobject	idNumber = 101 (set by Student(sId) via Person(101))
Employee subobject, containing a Person subobject	idNumber = 202 (set by Employee(eId) via Person(202))
TeachingAssistant's own members	none

Name lookup for idNumber finds one member in each Person subobject and cannot choose between them, so the program is ill-formed ([class.member.lookup]). Running a version that names the path explicitly

(ta.Student::idNumber and ta.Employee::idNumber) confirmed it: the Person constructor ran twice (Person initialized: 101, Person initialized: 202) and the two fields held 101 and 202. On this g++, sizeof(Person) is 4 and sizeof(TeachingAssistant) is 8, two ints. Writing ta.Student::idNumber would silence the error but keeps the duplicated data, which is the design flaw; virtual inheritance removes it.

Task 3: Refactoring with virtual base class inheritance

```
#include <iostream>
#include <string>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};

// Virtual inheritance: one shared Person subobject
class Student : virtual public Person {
public:
    Student(int id) : Person(id) {
        std::cout << "Student constructed" << std::endl;
    }
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {
        std::cout << "Employee constructed" << std::endl;
    }
};

class TeachingAssistant : public Student, public Employee {
public:
    // The most derived class initializes the virtual base directly
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {
        std::cout << "TeachingAssistant constructed" << std::endl;
    }
};

int main() {
    TeachingAssistant ta(101);
    std::cout << "TA ID: " << ta.idNumber << std::endl;   // unambiguous now
    return 0;
}
```

Student and Employee now say virtual public Person, so both share a single Person subobject. (The print statements in Student, Employee and TeachingAssistant were added to make the construction order visible, and TeachingAssistant takes one id because there is now only one Person.) With virtual inheritance the object also stores hidden links to the shared subobject: on this g++, sizeof(TeachingAssistant) grows to 24 bytes (implementation specific).

Task 4: Constructor delegation with virtual bases

The most derived class (TeachingAssistant) is solely responsible for initializing the virtual base, so TeachingAssistant's constructor is the one that invokes Person(int) ([class.base.init]). The Person(id) calls written in the initializer lists of Student and Employee are ignored when the object being built is a TeachingAssistant (they are used only when a plain Student or Employee is created). If TeachingAssistant omitted Person(id), the default constructor would run instead (idNumber = 0), and if Person had no default constructor it would not compile.

Construction order: virtual base classes first, then non-virtual bases in declaration order (Student, then Employee), then members, then the body of TeachingAssistant. Destruction runs in the reverse order.

Step	Action	Output
1	TeachingAssistant ta(101): memory for the whole object, containing one Person	
2	Virtual base first: Person(101) called directly by TeachingAssistant	Person initialized: 101
3	Student(101): its Person(id) is skipped (already built)	Student constructed
4	Employee(101): its Person(id) is skipped	Employee constructed
5	TeachingAssistant body runs	TeachingAssistant constructed
6	ta.idNumber: only one idNumber exists, so no ambiguity	TA ID: 101

Console output

```
Person initialized: 101
Student constructed
Employee constructed
TeachingAssistant constructed
TA ID: 101
```

Question 2: Missing Base Constructor in Derived Initializer List

Defect classification

Line	Defect	Category
18 (g++ line 21)	Sensor(std::string mac, int rate) has no Device(mac) in its member initializer list	Compilation error: no matching function for call to 'Device::Device()'
Body of Sensor constructor	macAddress = mac; assigns inside the body instead of initializing the base	Design error: assignment is too late, the base must already exist

Task 1: Why the error appears before the body runs

Before the body of a constructor executes, every base subobject and data member must already be initialized. Those not named in the member initializer list are default-initialized ([class.base.init]). Sensor's constructor names no base, so the compiler tries to call the implicit Device::Device(), the zero-argument constructor. Device declares only Device(std::string), and declaring any constructor suppresses the implicitly generated default constructor, so no such function exists and compilation fails. The error is detected at the opening of the Sensor constructor, so the body (macAddress = mac;) is irrelevant. Even if it compiled, the body would only assign to macAddress after the base had already been built.

Task 2: Chronological sequence of constructor execution

1. Virtual base classes (if any), in depth-first, left-to-right order.
2. Direct non-virtual base classes, in the order they are declared in the class definition (not in the initializer list).
3. Non-static data members, in the order they are declared in the class (again, not the initializer-list order).
4. The body of the derived class constructor.

For Sensor: Device(mac) runs first (printing its message), then samplingRate is initialized, then the body of Sensor. Destructors run in exactly the reverse order.

Task 3: Corrected constructor

```
Sensor(std::string mac, int rate) : Device(mac), samplingRate(rate) {
    std::cout << "Sensor configured with rate: " << samplingRate << " Hz" << std::endl;
}
```

The line printing the sensor rate in the body was added to make the order visible. Complete program:

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: " << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    // Base constructor called in the member initializer list
    Sensor(std::string mac, int rate) : Device(mac), samplingRate(rate) {
        std::cout << "Sensor configured with rate: " << samplingRate << " Hz" << std::endl;
    }

    void display() const {
        std::cout << "MAC: " << macAddress << ", Rate: " << samplingRate << " Hz" << std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Task 4: Dry run and verified output

Step	Action	Output
1	Sensor tempSensor("00:1A:2B:3C:4D:5E", 100)	
2	Device(mac) from the initializer list: macAddress is set	Device configured with MAC: 00:1A:2B:3C:4D:5E
3	samplingRate(rate): samplingRate = 100	
4	Sensor constructor body	Sensor configured with rate: 100 Hz
5	tempSensor.display()	MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz

Console output

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
Sensor configured with rate: 100 Hz
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Class Pointer Member Access & Object Slicing

Defect classification

Line	Defect	Category
22 (g++ line 24)	vPtr->engageTurbo() through a Vehicle*	Compilation error: 'class Vehicle' has no member named 'engageTurbo' (static type checking)
26	Vehicle plainVehicle = car;	Logic error: object slicing (compiles silently)
delete vPtr;	Vehicle has a non-virtual destructor but is deleted through a Vehicle* that points to a SportsCar	Undefined behavior ([expr.delete]); the g++ build gave no warning, so it is easy to miss

Task 1: Why Line 22 fails: static type checking

Member lookup is done at compile time, using the static type of the expression, not the type of the object at run time. The expression *vPtr has static type Vehicle, so the compiler looks for engageTurbo in Vehicle and its bases, and does not find it. It cannot know what vPtr will point to when the program runs. In memory the SportsCar object is [Vehicle part: maxSpeed][turboEnabled], and a Vehicle* provides a view of only the first part. The same early-binding rule means that a non-virtual honk() called through a Vehicle* always runs Vehicle::honk, even if SportsCar defined its own version.

Task 2: Object slicing on Line 26

Vehicle plainVehicle = car; creates a new, genuine Vehicle object by copy-constructing it from the Vehicle part of car (the copy constructor of Vehicle takes a const Vehicle&, which binds to that subobject). Only the base portion is copied: maxSpeed (250) is copied, but turboEnabled is simply dropped, since a Vehicle has no such member, and there is no way to get it back. The object's dynamic type is Vehicle, so with virtual functions the Vehicle versions are used, and polymorphism is lost. No error or warning is issued.

Task 3: Explicit downcast, and avoiding slicing

- **Downcast:** static_cast<SportsCar*>(vPtr)->engageTurbo(); is correct when the programmer knows the object really is a SportsCar. The safer form is dynamic_cast<SportsCar*>(vPtr), which returns a null pointer if the object is not a SportsCar; it requires the base class to be polymorphic (have a virtual function), which is why Vehicle was given a virtual destructor and a virtual honk().
- **Avoid slicing:** hold derived objects through a reference (Vehicle &r = car;) or a pointer (Vehicle *p = &car;), or smart pointers; both refer to the original complete object. Pass polymorphic objects to functions by reference or pointer, never by value.
- **Virtual destructor:** declare virtual ~Vehicle() so that delete vPtr destroys the whole SportsCar.

Task 4: Corrected code

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() {}                        // needed to delete a derived object via
Vehicle*
    virtual void honk() { std::cout << "General Horn" << std::endl; }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
```

```
    void honk() override { std::cout << "Sports Horn" << std::endl; }
    void engageTurbo() { std::cout << "Turbo Engaged!" << std::endl; }
};

int main() {
    Vehicle *vPtr = new SportsCar(300, true);   // upcast
    vPtr->honk();                               // virtual dispatch: Sports Horn

    // Explicit downcast to reach derived-only members
    static_cast<SportsCar*>(vPtr)->engageTurbo();

    // Safer, checked downcast (works because Vehicle is polymorphic)
    if (SportsCar *sp = dynamic_cast<SportsCar*>(vPtr)) {
        std::cout << "dynamic_cast OK, turbo = " << sp->turboEnabled << std::endl;
    }

    SportsCar car(250, true);

    Vehicle plainVehicle = car;                 // SLICED copy: turboEnabled lost
    plainVehicle.honk();

    Vehicle &vRef = car;                        // reference: no slicing
    vRef.honk();
    Vehicle *vPtr2 = &car;                      // pointer: no slicing
    vPtr2->honk();

    delete vPtr;                                // safe: virtual destructor
    return 0;
}
```

Additions: Vehicle::honk is virtual and SportsCar overrides it, so the difference between a pointer/reference and a sliced copy shows in the output.

Step	Statement	Result
1	vPtr = new SportsCar(300, true)	heap object: maxSpeed 300, turboEnabled true; vPtr has static type Vehicle*
2	vPtr->honk()	virtual call: runs SportsCar::honk, prints Sports Horn
3	static_cast<SportsCar*>(vPtr)->engageTurbo()	prints Turbo Engaged!
4	dynamic_cast<SportsCar*>(vPtr)	succeeds (the object really is a SportsCar): dynamic_cast OK, turbo = 1
5	Vehicle plainVehicle = car; plainVehicle.honk()	sliced copy: only maxSpeed = 250 copied; prints General Horn
6	vRef.honk() and vPtr2->honk()	refer to the real SportsCar: Sports Horn twice
7	delete vPtr	virtual destructor: SportsCar is destroyed correctly, no output

Console output

```
Sports Horn
Turbo Engaged!
dynamic_cast OK, turbo = 1
General Horn
Sports Horn
Sports Horn
```

Question 4: Incorrect Return Type in this Pointer Method Chaining

Defect classification

Line	Defect	Category
11	setWidth returns WindowBox* (return this;)	Root cause: wrong return type for a fluent interface
30 (g++ line 32)	box.setWidth(800).setHeight(600);	Compilation error: request for member 'setHeight' in 'box.WindowBox::setWidth(800)', which is of pointer type 'WindowBox*' (maybe you meant to use '->' ?)

Task 1: Why .setHeight(600) is rejected

The expression box.setWidth(800) is evaluated first, and its type is WindowBox*, a pointer. The dot operator requires its left operand to be an object (class type) ([expr.ref]); a pointer needs the arrow ->. So .setHeight(600) is applied to something that is not an object, and the compiler rejects it (and suggests the ->). The call would compile as box.setWidth(800)->setHeight(600), but mixing -> and . in one chain is awkward, so the methods should all return a reference.

Task 2: this versus *this

	this	*this
Type	WindowBox* (pointer; const WindowBox* inside a const member function)	WindowBox (the object itself)
Value / category	a prvalue: the address of the object on which the member function was called	an lvalue designating that same object
Use	this->width, return this;	return *this; with a return type of WindowBox& gives the caller an alias to the original object

If the return type were plain WindowBox, return *this would return a copy, and in box.setWidth(800).setHeight(600) the second call would modify a temporary copy, leaving box unchanged. Returning WindowBox& makes every call in the chain act on the same box.

Task 3: The fix

```
WindowBox& setWidth(int width) {
    this->width = width;      // this-> distinguishes the member from the parameter
    return *this;             // reference to the object itself
}
```

Note that this->width = width; is required here, since the parameter named width hides the member of the same name; plain width = width; would assign the parameter to itself.

Task 4: Complete code, dry run and output

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
```

```
        this->width = width;
        return *this;                // reference to the object itself
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);   // chaining works

    box.display();
    return 0;
}
```

Step	Action	State
1	WindowBox box;	width = 0, height = 0
2	box.setWidth(800)	this->width = 800; returns a reference to box
3	.setHeight(600) on that reference (the same box)	this->height = 600; returns a reference to box
4	box.display()	prints the result

Console output

```
Window: 800 x 600
```

Question 5: Private Inheritance Access Violations & Inaccessible Base Cast

Defect classification

Line	Defect	Category
24 (g++ line 27)	channel.closePort() called from main()	Compilation error: 'void RawSocket::closePort()' is inaccessible within this context
27 (g++ line 30)	RawSocket *socketPtr = &channel;	Compilation error: 'RawSocket' is an inaccessible base of 'SecureChannel'
13	class SecureChannel : private RawSocket	Root cause: private derivation hides the base interface

Task 1: What private inheritance does to accessibility

In private inheritance, all public and protected members of the base become private members of the derived class ([class.access.base]). So inside SecureChannel, openPort() and closePort() are usable (which is why establishEncryptedTunnel can call openPort), but to code outside the class they are private. main() is outside the class, so channel.closePort() is forbidden. Private inheritance models "implemented in terms of", not "is-a": the base is an implementation detail that clients may not use.

Task 2: Why the upcast on Line 27 fails; when upcasting is allowed

Converting a pointer to a derived class into a pointer to its base is allowed only if the base class is accessible at the point of conversion. With private inheritance, the base-class relationship is itself private, so the conversion is

allowed only inside the members and friends of SecureChannel. In main() it is not, hence 'RawSocket' is an inaccessible base of 'SecureChannel'. Allowing it would let clients use a SecureChannel through the RawSocket interface and bypass what the class chose to hide.

Inheritance	Where the upcast Derived* to Base* is allowed
public	everywhere (the usual is-a relationship)
protected	inside members and friends of the derived class and of classes derived from it
private	only inside members and friends of the derived class itself

Task 3: Two ways to make closePort() accessible

- **Approach A: using-declaration.** Writing using RawSocket::closePort; in the public section of SecureChannel makes only that member public again. openPort stays hidden, and the upcast is still illegal.
- **Approach B: public inheritance or composition.** Public inheritance (class SecureChannel : public RawSocket) makes all public base members public and makes the upcast legal; use it only if a SecureChannel really is a RawSocket. Composition keeps the interface hidden and forwards selected calls:

```
class SecureChannel {
    RawSocket socket;                       // "has-a" relationship
public:
    void establishEncryptedTunnel(int port) { socket.openPort(port); /* ... */ }
    void closePort() { socket.closePort(); } // explicit forwarding
};
```

Task 4: Corrected code using Approach A

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }
    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

class SecureChannel : private RawSocket {
public:
    // Approach A: access declaration re-exposes only closePort publicly
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);                         // legal inside the derived class
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();                        // now public through the using-declaration

    // RawSocket *socketPtr = &channel;         // removed: base is private, upcast is illegal
here
    return 0;
}
```

The illegal upcast line is removed (shown commented out), because under private inheritance it cannot be made legal from main().

Step	Statement	Result
1	channel.establishEncryptedTunnel(443)	calls openPort(443) inside the class (legal): Port 443 opened.; then Tunnel encrypted with TLS 1.3
2	channel.closePort()	the using-declaration made it public: Port closed.

Console output

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Section B: Pseudo-Code Writing Questions

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Task 1: Pseudo-code

```
CLASS Person
  PROTECTED:
    personID : INTEGER
    fullName : STRING
    email    : STRING
  PUBLIC:
    CONSTRUCTOR(id : INTEGER, name : STRING, mail : STRING)
ENDCLASS

CLASS Student INHERITS VIRTUAL Person
  PROTECTED:
    gpa           : REAL
    creditsEarned : INTEGER
  PUBLIC:
    CONSTRUCTOR(id, name, mail, g : REAL, credits : INTEGER) : Person(id, name, mail)
ENDCLASS

CLASS Faculty INHERITS VIRTUAL Person
  PROTECTED:
    academicDepartment : STRING
    rankTitle          : STRING
  PUBLIC:
    CONSTRUCTOR(id, name, mail, dept : STRING, rank : STRING) : Person(id, name, mail)
ENDCLASS

CLASS TeachingAssistant INHERITS Student, Faculty
  PRIVATE:
    stipendAmount  : REAL
    assignedCourse : STRING
  PUBLIC:
    // The most derived class initializes the virtual base Person DIRECTLY
    CONSTRUCTOR(id, name, mail, g, credits, dept, rank, stipend : REAL, course : STRING)
        : Person(id, name, mail),
          Student(id, name, mail, g, credits),
          Faculty(id, name, mail, dept, rank)
    METHOD DisplayTARecord()
ENDCLASS

CONSTRUCTOR TeachingAssistant::TeachingAssistant(...)
  stipendAmount  <- stipend
  assignedCourse <- course
END CONSTRUCTOR
```

```
METHOD TeachingAssistant::DisplayTARecord()
  // exactly ONE Person subobject, so these names are unambiguous
  OUTPUT "ID         : ", personID
  OUTPUT "Name       : ", fullName
  OUTPUT "GPA        : ", gpa
  OUTPUT "Department : ", academicDepartment
  OUTPUT "Stipend    : ", stipendAmount
END METHOD

PROCEDURE Main
  ta <- TeachingAssistant(2041, "Meera Nair", "meera.nair@univ.edu",
                          3.8, 96, "Computer Science", "Graduate Assistant",
                          18000, "CS101")
  ta.DisplayTARecord()
END PROCEDURE
```

Task 2: Trace of the instantiation

Step	Action	Effect
1	ta <- TeachingAssistant(2041, "Meera Nair", …)	one object is allocated holding a single shared Person subobject
2	Virtual base first: Person(2041, "Meera Nair", "meera.nair@univ.edu") called directly by the TA constructor	personID = 2041, fullName = Meera Nair, email set (once)
3	Student(...) : its Person(id, name, mail) call is ignored (already built)	gpa = 3.8, creditsEarned = 96
4	Faculty(...) : its Person call is ignored	academicDepartment = Computer Science, rankTitle = Graduate Assistant
5	TeachingAssistant body	stipendAmount = 18000, assignedCourse = CS101
6	ta.DisplayTARecord()	prints the record below
7	end of Main: destructors in reverse order	TA, then Faculty, then Student, and Person last

Console output

```
ID         : 2041
Name       : Meera Nair
GPA        : 3.8
Department : Computer Science
Stipend    : 18000
```

Without the keyword VIRTUAL, TeachingAssistant would contain two Persons, and personID and fullName would be ambiguous in DisplayTARecord.

Question 2: Polymorphic Payroll with Abstract Base Class & Base Pointers

Task 1: Pseudo-code

```
ABSTRACT CLASS Employee
  PROTECTED:
    employeeID : STRING
    name       : STRING
  PUBLIC:
    CONSTRUCTOR(id : STRING, n : STRING)
    VIRTUAL DESTRUCTOR()                          // needed to DEALLOCATE through Employee*
    ABSTRACT METHOD CalculatePay() RETURNS REAL  // pure virtual: no body
    METHOD DisplayHeader()
ENDCLASS
```

```
CONSTRUCTOR Employee::Employee(id, n)
  employeeID <- id
  name       <- n
END CONSTRUCTOR

METHOD Employee::DisplayHeader()
  OUTPUT employeeID, " - ", name
END METHOD

CLASS SalariedEmployee EXTENDS Employee
  PRIVATE:
    monthlySalary : REAL
  PUBLIC:
    CONSTRUCTOR(id, n, salary : REAL) : Employee(id, n)
    METHOD CalculatePay() RETURNS REAL OVERRIDE
ENDCLASS

CONSTRUCTOR SalariedEmployee::SalariedEmployee(id, n, salary) : Employee(id, n)
  monthlySalary <- salary
END CONSTRUCTOR

METHOD SalariedEmployee::CalculatePay() RETURNS REAL
  RETURN monthlySalary
END METHOD

CLASS HourlyEmployee EXTENDS Employee
  PRIVATE:
    hourlyWage  : REAL
    hoursWorked : REAL
  PUBLIC:
    CONSTRUCTOR(id, n, wage : REAL, hours : REAL) : Employee(id, n)
    METHOD CalculatePay() RETURNS REAL OVERRIDE
ENDCLASS

CONSTRUCTOR HourlyEmployee::HourlyEmployee(id, n, wage, hours) : Employee(id, n)
  hourlyWage  <- wage
  hoursWorked <- hours
END CONSTRUCTOR

METHOD HourlyEmployee::CalculatePay() RETURNS REAL
  IF hoursWorked <= 40 THEN
    RETURN hoursWorked * hourlyWage
  ELSE
    regular  <- 40 * hourlyWage
    overtime <- (hoursWorked - 40) * hourlyWage * 1.5
    RETURN regular + overtime
  ENDIF
END METHOD

PROCEDURE Main
  staff <- ALLOCATE ARRAY[2] OF POINTER TO Employee
  staff[0] <- NEW SalariedEmployee("E101", "Asha", 8500)
  staff[1] <- NEW HourlyEmployee("H202", "Ravi", 50, 48)
  total <- 0.0
  FOR i <- 0 TO 1
    staff[i]->DisplayHeader()
    pay <- staff[i]->CalculatePay()             // late binding: the real class decides
    OUTPUT "Pay: ", pay
    total <- total + pay
  NEXT i
  OUTPUT "Total disbursement: ", total
  FOR i <- 0 TO 1
    DEALLOCATE staff[i]                         // virtual destructor frees the derived object
  NEXT i
  DEALLOCATE staff
END PROCEDURE
```

An ABSTRACT CLASS cannot be instantiated (NEW Employee(...) would be an error); only the concrete subclasses that override CalculatePay() can.

Task 2: Dry-run trace

i	Object / dynamic type	CalculatePay() evaluation	Running total
0	staff[0] -> SalariedEmployee	returns monthlySalary = 8,500	0 + 8,500 = 8,500
1	staff[1] -> HourlyEmployee (50/h, 48 h)	hours > 40, so regular = 40 x 50 = 2,000; overtime = (48 - 40) x 50 x 1.5 = 8 x 75 = 600; pay = 2,600	8,500 + 2,600 = 11,100

Output: E101 - Asha, Pay: 8500; H202 - Ravi, Pay: 2600; Total disbursement: 11100.

Question 3: Composite Canvas Engine with Member Classes & this Chaining

Task 1: Pseudo-code

```
CLASS Point2D
  PRIVATE:
    x : REAL
    y : REAL
  PUBLIC:
    CONSTRUCTOR(a : REAL = 0.0, b : REAL = 0.0)
    METHOD GetX() RETURNS REAL
    METHOD GetY() RETURNS REAL
ENDCLASS

CONSTRUCTOR Point2D::Point2D(a, b)
  x <- a
  y <- b
END CONSTRUCTOR

CLASS Circle
  PRIVATE:
    centerPoint : Point2D          // member object ("has-a")
    radius      : REAL
  PUBLIC:
    CONSTRUCTOR()
    METHOD SetCenter(newX : REAL, newY : REAL) RETURNS REF Circle
    METHOD SetRadius(r : REAL) RETURNS REF Circle
    METHOD Draw()
ENDCLASS

CONSTRUCTOR Circle::Circle() : centerPoint(0.0, 0.0), radius(1.0)
  // member initializer list builds the Point2D BEFORE this body runs
END CONSTRUCTOR

METHOD Circle::SetCenter(newX, newY) RETURNS REF Circle
  centerPoint <- Point2D(newX, newY)
  RETURN REF THIS
END METHOD

METHOD Circle::SetRadius(r) RETURNS REF Circle
  IF r > 0 THEN
    radius <- r
  ELSE
    OUTPUT "Invalid radius, value ignored"
  ENDIF
  RETURN REF THIS
END METHOD
```

```
METHOD Circle::Draw()
  OUTPUT "Circle: center (", centerPoint.GetX(), ", ", centerPoint.GetY(),
         "), radius ", radius
END METHOD

PROCEDURE Main
  myCircle <- Circle()
  myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
END PROCEDURE
```

Task 2: Trace of myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()

Step	Call	State of myCircle	Value passed on
0	Circle() constructed: Point2D(0, 0) built first, radius = 1.0	center (0, 0), radius 1.0	-
1	myCircle.SetCenter(12.5, 30.0)	center (12.5, 30.0), radius 1.0	REF THIS, an alias for myCircle
2	(that reference).SetRadius(7.5): 7.5 > 0 is true	center (12.5, 30.0), radius 7.5	REF THIS, still myCircle
3	(that reference).Draw()	unchanged	-

Console output

```
Circle: center (12.5, 30.0), radius 7.5
```

Because each setter returns a reference to the original object, not a copy, all three calls act on myCircle itself. The member object centerPoint is constructed before the Circle body and destroyed after it (reverse order).

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Task 1: Pseudo-code

```
CLASS Vehicle
  PROTECTED:
    vehicleVIN   : STRING
    baseWeightKg : REAL
  PUBLIC:
    CONSTRUCTOR(vin : STRING, weight : REAL)
    VIRTUAL DESTRUCTOR()                        // needed to DEALLOCATE via Vehicle*
    METHOD DisplaySpecs()
ENDCLASS

CONSTRUCTOR Vehicle::Vehicle(vin, weight)
  vehicleVIN   <- vin
  baseWeightKg <- weight
  OUTPUT "Vehicle constructed"
END CONSTRUCTOR

METHOD Vehicle::DisplaySpecs()
  OUTPUT "VIN: ", vehicleVIN, "  Base weight: ", baseWeightKg, " kg"
END METHOD

CLASS AutonomousVehicle EXTENDS Vehicle
  PROTECTED:
    lidarSensorID   : STRING
    firmwareVersion : STRING
  PUBLIC:
    CONSTRUCTOR(vin, weight, lidar : STRING, fw : STRING) : Vehicle(vin, weight)
ENDCLASS

CONSTRUCTOR AutonomousVehicle::AutonomousVehicle(vin, weight, lidar, fw) : Vehicle(vin, weight)
```

```
  lidarSensorID   <- lidar
  firmwareVersion <- fw
  OUTPUT "AutonomousVehicle constructed"
END CONSTRUCTOR

CLASS DeliveryDrone EXTENDS AutonomousVehicle
  PRIVATE:
    payloadCapacityKg   : REAL
    batteryChargePercent : REAL
  PUBLIC:
    CONSTRUCTOR(vin, weight, lidar, fw, payload : REAL, battery : REAL)
        : AutonomousVehicle(vin, weight, lidar, fw)
    METHOD DisplayFlightStatus()
ENDCLASS

CONSTRUCTOR DeliveryDrone::DeliveryDrone(vin, weight, lidar, fw, payload, battery)
    : AutonomousVehicle(vin, weight, lidar, fw)
  payloadCapacityKg    <- payload
  batteryChargePercent <- battery
  OUTPUT "DeliveryDrone constructed"
END CONSTRUCTOR

METHOD DeliveryDrone::DisplayFlightStatus()
  DisplaySpecs()                                        // inherited from Vehicle
  OUTPUT "LiDAR: ", lidarSensorID, "  Firmware: ", firmwareVersion
  OUTPUT "Payload: ", payloadCapacityKg, " kg  Battery: ", batteryChargePercent, "%"
END METHOD

PROCEDURE Main
  DECLARE vPtr AS POINTER TO Vehicle
  vPtr <- NEW DeliveryDrone("VIN-DR-7781", 12.5, "LDR-45", "v3.2.1", 5.0, 87)
  vPtr->DisplaySpecs()                                  // only Vehicle members visible

  DECLARE dPtr AS POINTER TO DeliveryDrone
  dPtr <- CAST vPtr TO POINTER TO DeliveryDrone         // downcast for drone telemetry
  dPtr->DisplayFlightStatus()

  DEALLOCATE vPtr                                       // destructors run leaf to root
END PROCEDURE
```

Task 2: Constructor invocation order (root to leaf)

The statement NEW DeliveryDrone(...) calls the DeliveryDrone constructor first, but its initializer list immediately delegates up the chain. Each constructor body runs only after its base class constructor has finished, so execution reaches the root before any body runs.

Step	Call	Result
1	DeliveryDrone(...) entered; its list calls AutonomousVehicle(vin, weight, lidar, fw)	nothing printed yet
2	AutonomousVehicle entered; its list calls Vehicle(vin, weight)	nothing printed yet
3	Vehicle body runs (root)	vehicleVIN = VIN-DR-7781, baseWeightKg = 12.5; prints Vehicle constructed
4	AutonomousVehicle body runs	lidarSensorID = LDR-45, firmwareVersion = v3.2.1; prints AutonomousVehicle constructed
5	DeliveryDrone body runs (leaf)	payload = 5.0, battery = 87; prints DeliveryDrone constructed

Console output of the construction: Vehicle constructed, AutonomousVehicle constructed, DeliveryDrone constructed. DEALLOCATE vPtr runs the destructors in the reverse order: DeliveryDrone, AutonomousVehicle, Vehicle. This works through a Vehicle* only because the destructor is VIRTUAL.

Telemetry: vPtr->DisplaySpecs() prints VIN: VIN-DR-7781 Base weight: 12.5 kg. After the cast, dPtr->DisplayFlightStatus() prints those specs, then LiDAR: LDR-45 Firmware: v3.2.1 and Payload: 5.0 kg Battery: 87%. The cast is valid because the object really is a DeliveryDrone; vPtr alone could not call DisplayFlightStatus (static type is Vehicle*).

Question 5: Dynamic Object Pointer Array & Zero-Copy Sorting Engine

Task 1: Pseudo-code

```
CLASS Product
  PRIVATE:
    sku        : STRING
    price      : REAL
    stockCount : INTEGER
  PUBLIC:
    CONSTRUCTOR(s : STRING, p : REAL, stock : INTEGER)
    METHOD PrintProduct()
    METHOD GetPrice() RETURNS REAL
ENDCLASS

CONSTRUCTOR Product::Product(s, p, stock)
  sku <- s
  price <- p
  stockCount <- stock
END CONSTRUCTOR

METHOD Product::PrintProduct()
  OUTPUT sku, "  price: ", price, "  stock: ", stockCount
END METHOD

PROCEDURE SortByPrice(catalog : ARRAY OF POINTER TO Product, n : INTEGER)
  FOR i <- 0 TO n - 2
    FOR j <- 0 TO n - 2 - i
      IF catalog[j]->GetPrice() > catalog[j+1]->GetPrice() THEN   // compare through the
pointers
        temp         <- catalog[j]                                // swap the POINTERS only;
        catalog[j]   <- catalog[j+1]                              // the Product objects
        catalog[j+1] <- temp                                      // never move in memory
      ENDIF
    NEXT j
  NEXT i
END PROCEDURE

PROCEDURE Main
  n <- 3
  DECLARE catalog AS ARRAY[n] OF POINTER TO Product
  catalog[0] <- NEW Product("LAP-01", 1200.0, 15)
  catalog[1] <- NEW Product("MOU-02",   25.0, 120)
  catalog[2] <- NEW Product("MON-03",  350.0, 40)

  SortByPrice(catalog, n)

  FOR k <- 0 TO n - 1
    catalog[k]->PrintProduct()
  NEXT k

  FOR k <- 0 TO n - 1
    DEALLOCATE catalog[k]                 // free every object exactly once
    catalog[k] <- NULL
  NEXT k
```

```
END PROCEDURE
```

The comparison uses catalog[j]->price (written here through GetPrice() because price is private; inside a member or friend it could be read directly). Only the pointer variables are exchanged, so each swap moves one machine address, not a whole Product (sku, price and stock).

Task 2: Pointer-swap trace (Laptop 1200, Mouse 25, Monitor 350)

The objects live at fixed heap addresses: Laptop at @A1, Mouse at @A2, Monitor at @A3. They never move; only the contents of catalog[] change.

Pass	j	Comparison (through ->)	Swap?	catalog[0], catalog[1], catalog[2] afterwards
-	-	initial	-	@A1 Laptop 1200, @A2 Mouse 25, @A3 Monitor 350
1	0	catalog[0]->price 1200 > catalog[1]->price 25	YES	@A2 Mouse, @A1 Laptop, @A3 Monitor
1	1	catalog[1]->price 1200 > catalog[2]->price 350	YES	@A2 Mouse, @A3 Monitor, @A1 Laptop
2	0	catalog[0]->price 25 > catalog[1]->price 350 ? false	NO	@A2 Mouse, @A3 Monitor, @A1 Laptop

Two passes and two pointer swaps. Final catalog order (ascending price): Mouse 25, Monitor 350, Laptop 1200.

Console output

```
MOU-02  price: 25  stock: 120
MON-03  price: 350  stock: 40
LAP-01  price: 1200  stock: 15
```

Finally the loop DEALLOCATE catalog[k] frees the three objects (@A2, @A3, @A1), each once.